



UNIVERSITAT
Carlemany

Formación europea
para desafíos reales

De:

 Planeta Formación y Universidades

Bloque 2- Introducción al Testing

Aplicaciones basadas en tecnologías web

Bloque 2 — Introducción al Testing

- Por qué se testea en backend
- Qué es un test unitario
- Proyecto de tests en Visual Studio
- Framework de testing en .NET
- Anatomía de un test: Arrange, Act, Assert
- Qué se testea y qué NO en backend
- Test unitario real sobre lógica de negocio
- Tests orientados a comportamiento (no a implementación)
- Tests aunque la implementación sea dummy
- Pruebas manuales con Postman
- Qué mirar en Postman (más allá del body)

Por qué se testea en backend

Problema real

Cambias una lógica “pequeña” y semanas después alguien detecta que una métrica ya no cuadra.

Ejemplo:

- Antes: el cálculo de exposición devuelve 150000
- Después de un cambio: devuelve 0
- Nadie se da cuenta hasta que el dato llega a producción

 Un **test unitario** detecta el error **en el momento del cambio**, no semanas después.

 *El testing backend no evita errores: evita errores silenciosos.*

Qué es un test unitario (en la práctica)

Un **test unitario** verifica:

- una **unidad pequeña de lógica**
- con **inputs conocidos**
- y **outputs esperados**

Ejemplo de lógica a testear:

```
public int CalculateExposure(int value, int factor)
{
    return value * factor;
}
```

Test unitario asociado:

```
Assert.Equal(100, CalculateExposure(50, 2));
```

Un test unitario NO:

- levanta servidores
- accede a BD
- usa HTTP

 *Si necesitas Postman, no es un test unitario.*

Proyecto de tests en Visual Studio

En .NET, los tests viven en **un proyecto separado**.

Estructura típica:

Solution

```
├─ BackendApp
└─ BackendApp.Tests
```

En Visual Studio:

- botón derecho → *Add* → *New Project*
- tipo: **xUnit Test Project**
- referencia al proyecto principal

Ejecutar tests:

- Test Explorer
- botón  *Run All*

 *Separar tests del código productivo es parte del diseño.*

Framework de testing en .NET (xUnit)

xUnit es uno de los frameworks más usados en .NET.

Ejemplo de test:

```
public class ExposureTests
{
    [Fact]
    public void CalculateExposure_ReturnsExpectedValue()
    {
        var result = CalculateExposure(50, 2);
        Assert.Equal(100, result);
    }
}
```

Qué significa:

- [Fact] → esto es un test
- Assert.Equal → comparación esperada

Visual Studio:

- muestra verde  si pasa
- rojo  si falla

 *Un test fallido es información, no un problema.*

Anatomía de un test: Arrange, Act, Assert

Todo test unitario sigue este patrón mental:

Arrange — preparar

- `var service = new ExposureService();`

Act — ejecutar

- `var result = service.GetTotalExposure();`

Assert — verificar

- `Assert.Equal(150000, result);`

Ventaja:

- el test se lee como una historia
- es fácil detectar qué falla

 *Un buen test se entiende sin comentarios.*

Qué se testea y qué NO en backend

Se testea:

- servicios de negocio
- cálculos
- reglas
- transformaciones (ETL)

Ejemplo:

ExposureService

No se testea (aquí):

- Controllers
- HttpClient
- Postman
- infraestructura

Ejemplo:

AssetsController // no aquí

 *Se testea el “cerebro”, no la “puerta de entrada”.*

Test unitario real sobre lógica de negocio

Caso real

Un servicio calcula si un activo es de **alto riesgo** según su valor.

Código a testear:

```
public bool IsHighRisk(decimal value)
{
    return value > 100000;
}
```

Test unitario:

```
[Fact]
public void IsHighRisk_WhenValueIsHigh_ReturnsTrue()
{
    var result = IsHighRisk(150000);
    Assert.True(result);
}
```

Qué demuestra el test:

- entrada clara
- resultado esperado
- comportamiento explícito

 *El test valida una regla de negocio concreta.*

Tests orientados a comportamiento (no a implementación)

Un buen test **describe lo que debe pasar**, no cómo está hecho el código.

Mal test (acoplado):

```
Assert.Equal(3, internalList.Count);
```

Buen test (comportamiento):

```
Assert.True(result.IsHighRisk);
```

Lectura del test:

“Dado un valor alto, el activo debe considerarse de alto riesgo”

 *Si cambias la implementación y el test sigue pasando, el test es bueno.*

Tests aunque la implementación sea dummy

A veces la lógica aún no está terminada.

Implementación dummy:

```
public decimal GetTotalExposure()  
{  
    return 100000;  
}
```

Test unitario:

```
[Fact]  
public void GetTotalExposure_ReturnsExpectedValue()  
{  
    var result = service.GetTotalExposure();  
    Assert.Equal(100000, result);  
}
```

Qué aporta el test:

- define el comportamiento esperado
- actúa como contrato
- podrá evolucionar con la implementación

 *El test es válido aunque el código sea provisional.*

Pruebas manuales con Postman (caso real)

Caso: comprobar que un endpoint funciona.

Petición:

GET /assets/total-exposure

En Postman se valida:

- método HTTP
- URL correcta
- código de estado (200 OK)
- body esperado

Ejemplo de respuesta:

```
{ "totalExposure": 150000 }
```

 *Postman valida el contrato HTTP, no la lógica interna.*

Qué mirar en Postman (más allá del body)

No solo importa el JSON.

Revisar siempre:

- status code (200, 400, 404)
- headers
- tiempo de respuesta
- errores coherentes

Ejemplo de error correcto:

404 Not Found

Ejemplo incorrecto:

200 OK

```
{ "error": "not found" }
```

 *El código HTTP comunica antes que el cuerpo*